Backpropagation in One MLP Layer: From a Three-Dimensional Tensor to an Outer Product

N-20250926

§1.1 The setting

Consider one middle layer of a multilayer perceptron:

$$z = Wx + b, \qquad y = \sigma(z).$$

Here

$$x \in \mathbb{R}^n, \qquad W \in \mathbb{R}^{m \times n}, \qquad b \in \mathbb{R}^m, \qquad z, y \in \mathbb{R}^m.$$

The activation function σ acts elementwise, for example sigmoid or ReLU. The goal of backpropagation is to compute

$$\frac{\partial L}{\partial W}$$

for a loss function L.

The softmax–cross-entropy setting provides a standard example. If the final layer produces logits z_i, then

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad L = -\sum_i t_i \ln \hat{y}_i,$$

where t_i is a one-hot label. Backpropagation asks how this scalar loss changes when each weight $W_{j,k}$ changes.

§1.2 The theoretical tensor

Strictly speaking, the derivative of the vector $y \in \mathbb{R}^m$ with respect to the matrix $W \in \mathbb{R}^{m \times n}$ has three indices:

$$\frac{\partial y_i}{\partial W_{j,k}}.$$

The index i chooses the output component, j chooses the row of W, and k chooses the column of W. So the object has shape

$$m \times m \times n.$$

This is the source of the confusion recorded in the note. The derivative is theoretically a third-order tensor, but in neural-network code one usually sees only a matrix gradient. The matrix appears after chain-rule contraction with the upstream gradient.

§1.3 Computing the sparse derivative

Write the linear preactivation component as

$$z_r = \sum_{t=1}^{n} W_{r,t} x_t + b_r.$$

Then

$$\frac{\partial z_r}{\partial W_{j,k}} = \begin{cases} x_k, & r = j, \\ 0, & r \neq j, \end{cases} = \delta_{rj} x_k.$$

This is sparse because z_r depends only on row r of W.

For $y_i = \sigma(z_i)$ with elementwise activation,

$$\frac{\partial y_i}{\partial z_r} = 0 \quad (i \neq r), \qquad \frac{\partial y_i}{\partial z_i} = \sigma'(z_i).$$

Thus

$$\frac{\partial y_i}{\partial W_{j,k}} = \sum_{r=1}^{m} \frac{\partial y_i}{\partial z_r} \frac{\partial z_r}{\partial W_{j,k}} = \frac{\partial y_i}{\partial z_j} x_k.$$

For elementwise activation this is nonzero only when $i = j$.

§1.4 Contraction by the chain rule

The loss L is scalar. Therefore

$$\frac{\partial L}{\partial W_{j,k}} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial W_{j,k}}.$$

Substituting the previous formula gives

$$\frac{\partial L}{\partial W_{j,k}} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial z_j} x_k.$$

The sum over i is exactly the j-th component of the upstream gradient with respect to z:

$$\delta_j = \frac{\partial L}{\partial z_j}.$$

Hence

$$\frac{\partial L}{\partial W_{j,k}} = \delta_j x_k.$$

Stacking all j, k gives the outer product

$$\boxed{\frac{\partial L}{\partial W} = \delta x^T}$$

where

$$\delta = \frac{\partial L}{\partial z}.$$

§1.5 Why the tensor disappears

The original derivative $\partial y/\partial W$ is a tensor with shape (m, m, n). But backpropagation does not need to store it. The chain rule contracts it with the vector $\partial L/\partial y$, leaving a matrix of shape (m, n).

Equivalently:

$$\text{three-dimensional tensor} \xrightarrow{\text{chain-rule summation}} \text{matrix gradient.}$$

This is not a loss of information for gradient descent, because gradient descent only needs the derivative of the scalar loss with respect to each parameter.

§1.6 Contravariant reading of the same calculation

The same computation can be described as a pullback of covectors. A forward layer is a map

$$f : \text{parameters and activations} \longrightarrow \text{outputs.}$$

Its differential sends small forward perturbations to small output perturbations:

$$df : T_x X \to T_{f(x)} Y.$$

But the loss supplies a covector at the output,

$$dL \in T^*_{f(x)} Y,$$

and backpropagation sends it backward by precomposition:

$$f^*(dL) = dL \circ df \in T^*_x X.$$

This is why reverse-mode differentiation is naturally contravariant.

For a batched linear layer in PyTorch convention,

$$X \in \mathbb{R}^{b\times n}, \qquad W \in \mathbb{R}^{m\times n}, \qquad Y = XW^T \in \mathbb{R}^{b\times m},$$

the component formula is

$$Y_{\alpha i} = \sum_{k=1}^{n} X_{\alpha k} W_{ik}.$$

Hence

$$\frac{\partial Y_{\alpha i}}{\partial W_{jk}} = \delta_{ij} X_{\alpha k},$$

a Jacobian tensor of shape (b, m, m, n). Contracting with the upstream gradient $G = \partial L/\partial Y$, of shape (b, m), gives

$$\begin{aligned}\frac{\partial L}{\partial W_{jk}} &= \sum_{\alpha,i} G_{\alpha i} \frac{\partial Y_{\alpha i}}{\partial W_{jk}} \\ &= \sum_{\alpha} G_{\alpha j} X_{\alpha k},\end{aligned}$$

so

$$\frac{\partial L}{\partial W} = G^T X.$$

If the weights are stored instead as $W_{\text{col}} \in \mathbb{R}^{n\times m}$ and $Y = XW_{\text{col}}$, the same formula appears as

$$\frac{\partial L}{\partial W_{\text{col}}} = X^T G.$$

The two expressions differ only by the convention for storing the weight matrix.

§1.7 Outer-product expression

Let

$$\frac{\partial y}{\partial z}$$

be treated as a column vector for elementwise activation, with entries $\sigma'(z_i)$. Then the derivative of y with respect to W can be represented row by row as

$$\frac{\partial y}{\partial W} \sim \begin{bmatrix} \frac{\partial y_1}{\partial z_1} x^T \\ \frac{\partial y_2}{\partial z_2} x^T \\ \vdots \\ \frac{\partial y_m}{\partial z_m} x^T \end{bmatrix} = \left(\frac{\partial y}{\partial z}\right) x^T.$$

For the scalar loss, replace $\partial y/\partial z$ by the backpropagated vector $\delta = \partial L/\partial z$.

§1.8 Numerical example

The note gives a simple example with

$$m = 2, \qquad n = 3, \qquad x = [1, 2, 3]^T, \qquad z = [0.1, -1.0]^T.$$

For sigmoid,

$$\sigma'(0.1) \approx 0.2494, \qquad \sigma'(-1.0) \approx 0.1966.$$

Therefore

$$\frac{\partial y}{\partial W} \sim \begin{bmatrix} 0.2494 \\ 0.1966 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 0.2494 & 0.4988 & 0.7482 \\ 0.1966 & 0.3932 & 0.5898 \end{bmatrix}.$$

Each row is the input vector scaled by the derivative of the corresponding output.

§1.9 Einstein notation proof

An index calculation makes the chain rule explicit. Let $y = f(z)$ and $z = Wx$. For a matrix entry $A_{pq} = W_{pq}$,

$$\frac{\partial z_i}{\partial A_{pq}} = \frac{\partial}{\partial A_{pq}} \sum_j A_{ij} x_j = \delta_{ip} x_q.$$

Then

$$\frac{\partial L}{\partial A_{pq}} = \sum_i \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial z_p} x_q.$$

For elementwise activation, this becomes

$$\frac{\partial L}{\partial A_{pq}} = \frac{\partial L}{\partial z_p} x_q.$$

This is the component form of $\partial L/\partial W = \delta x^T$.

§1.10 Kronecker product viewpoint

The note also explores vectorization. If $y = f(Wx)$ and we vectorize W, then the Jacobian with respect to $\mathrm{vec}(W)$ can be written using Kronecker products. The important identity is that changing one matrix entry affects only one row of z, which is encoded by a Kronecker delta.

In batch form, if X is a matrix of inputs and Δ is the matrix of upstream gradients, then the gradient becomes

$$\frac{\partial L}{\partial W} = \Delta X^T.$$

This is the same outer-product rule, summed over the batch dimension.

§1.11 Softmax–cross-entropy simplification

For the final layer with softmax and cross entropy, the derivative simplifies to

$$\frac{\partial L}{\partial z} = \hat{y} - t.$$

Therefore the final-layer weight gradient is

$$\frac{\partial L}{\partial W} = (\hat{y} - t)x^T.$$

This is why implementation code often looks much simpler than the theoretical tensor derivation.

§1.12 A wider interpretation

The key lesson is that matrix calculus is compressed tensor calculus. The full derivative of a vector with respect to a matrix is a higher-order object, but backpropagation contracts it immediately with an upstream gradient. The result is an outer product: error signal times input. This is the local rule behind every dense neural-network layer.

§1.13 Backpropagation as cotangent pullback

For a layer $z = Wx + b$, the forward map sends activations forward, while backpropagation sends covectors backward:

$$\bar{x} = W^T \bar{z}.$$

This is the pullback on cotangent vectors.

§1.14 Why the three-dimensional tensor collapses

The derivative of $z_i = \sum_j W_{ij}x_j + b_i$ with respect to W_{ab} is

$$\frac{\partial z_i}{\partial W_{ab}} = \delta_{ia}x_b.$$

Contracting with upstream gradient $\bar{z}_i$ gives

$$\frac{\partial L}{\partial W_{ab}} = \bar{z}_a x_b.$$

Thus the huge derivative tensor disappears after contraction, leaving an outer product.

§1.15 Batch form

For a batch, gradients add over examples:

$$\nabla_W L = \bar{Z} X^T.$$

This is matrix multiplication as accumulated outer products.

§1.16 Backpropagation as sparse tensor contraction

Backpropagation is not magic tensor cancellation. It is cotangent pullback plus contraction of sparse Jacobians, and the familiar weight gradient is the outer product of upstream error with input activation.